

DAT12-3

Pandas I

Manipulating tabular data in Python

From a raw CSV to a clean, summarized DataFrame.

Albert School — 16 hours

Contents

1 DataFrames and Series	2
1.1 Loading data	2
1.2 Inspecting a DataFrame	2
2 Selection and filtering	3
2.1 loc and iloc	3
2.2 Boolean indexing	3
3 Missing values	3
3.1 Identifying	3
3.2 Dropping	3
3.3 Imputing	3
4 Groupby and aggregation	3
5 Merging and concatenating	4
5.1 Concatenating	4
5.2 Merging	4
6 Reshaping	4
7 Applying functions	4
8 End-to-end pipeline	4
9 Descriptive reporting	5

1 DataFrames and Series

pandas is a Python library for tabular data. Import it with the conventional alias:

```
import pandas as pd
```

A **Series** is a one-dimensional labelled array. A **DataFrame** is a two-dimensional labelled table: a collection of Series sharing one index. A column of a DataFrame is a Series.

1.1 Loading data

A DataFrame is loaded from a file with the matching reader:

```
df = pd.read_csv("data.csv")
df = pd.read_excel("data.xlsx")
df = pd.read_json("data.json")
```

1.2 Inspecting a DataFrame

The following attributes and methods report on the contents of a DataFrame:

- `df.shape` — a tuple (rows, columns).
- `df.dtypes` — the data type of each column.
- `df.head()` — the first five rows.
- `df.info()` — column names, non-null counts, and dtypes.
- `df.describe()` — count, mean, standard deviation, minimum, quartiles, and maximum for each numeric column.

```
df.shape          # (1000, 6)
df.dtypes
df.head()
df.info()
df.describe()
```

Each output is read as follows: `shape` gives the size of the table; `dtypes` tells whether a column is numeric, textual, or boolean; `head` shows sample values; `info` reveals how many values are missing per column; `describe` summarizes the distribution of each numeric column.

2 Selection and filtering

Columns are selected by name:

```
df["age"]          # one column, as a Series
df[["age", "city"]] # several columns, as a DataFrame
```

2.1 loc and iloc

`loc` selects by label; `iloc` selects by integer position. Both take a row selector and an optional column selector.

```
df.loc[0, "age"]      # by label
df.loc[0:5, ["age", "city"]]
df.iloc[0, 1]         # by position
df.iloc[0:5, 0:2]
```

2.2 Boolean indexing

A condition on a column produces a boolean Series, which selects the rows where it is true:

```
df[df["age"] > 30]
df[(df["age"] > 30) & (df["city"] == "Paris")]
```

Combine conditions with `&` (and) and `|` (or); each condition must be parenthesized.

3 Missing values

Missing entries are represented by `NaN`.

3.1 Identifying

```
df.isna()          # boolean DataFrame
df.isna().sum()     # count of missing values per column
```

3.2 Dropping

```
df.dropna()        # drop rows containing any NaN
df.dropna(axis=1)   # drop columns containing any NaN
```

3.3 Imputing

Missing values are replaced with `fillna`. The replacement value must be justified by the column:

```
df["age"].fillna(df["age"].mean()) # numeric: mean or median
df["city"].fillna("Unknown")      # categorical: a constant or the mode
```

State the strategy chosen and the reason for it: for example, impute a numeric column with its median when the distribution is skewed, and with its mean otherwise.

4 Groupby and aggregation

`groupby` splits the rows into groups by the values of one or more columns; an aggregation then reduces each group to a single value.

```
df.groupby("city")["sales"].mean()
df.groupby("city").agg({"sales": "sum", "age": "mean"})
df.groupby(["city", "year"])["sales"].sum()
```

Grouping by several columns summarizes along several dimensions at once.

5 Merging and concatenating

5.1 Concatenating

concat stacks DataFrames:

```
pd.concat([df1, df2])          # stack rows
pd.concat([df1, df2], axis=1)  # stack columns
```

5.2 Merging

merge joins two DataFrames on a shared key. The how argument selects the join type:

```
pd.merge(left, right, on="id", how="inner")
```

- **inner** — keep only keys present in both DataFrames.
- **left** — keep all keys of the left DataFrame; fill missing right values with NaN.
- **right** — keep all keys of the right DataFrame.
- **outer** — keep all keys of either DataFrame.

Choose the join by which rows must be retained: an inner join when only matched rows are wanted, a left join when every left row must be kept.

6 Reshaping

Data is held in **wide** format (one column per variable) or **long** format (one row per observation).

pivot turns long into wide:

```
df.pivot(index="date", columns="city", values="sales")
```

melt turns wide into long:

```
df.melt(id_vars="date", var_name="city", value_name="sales")
```

7 Applying functions

apply runs a function on each element, column, or row. It is commonly paired with a lambda expression.

```
df["age"].apply(lambda x: x + 1)          # on a column
df.apply(lambda row: row["a"] + row["b"], axis=1)  # on each row
```

8 End-to-end pipeline

The operations above combine into a pipeline from a raw file to a clean, summarized DataFrame:

```
df = pd.read_csv("sales.csv")          # load
df.info()                              # inspect
df = df.dropna(subset=["sales"])        # handle missing values
df["sales"] = df["sales"].fillna(df["sales"].median())
summary = (                             # summarize
    df.groupby("city")["sales"]
      .sum()
      .reset_index()
)
```

9 Descriptive reporting

A descriptive statistics report ends with a written interpretation of the findings. After producing the summary, write a paragraph that states, in plain language, what the numbers say: which groups are largest or smallest, where the data is concentrated, and any notable gaps or outliers. The paragraph reports what the figures show; it does not infer causes beyond the data.

Exercises

1. Load a CSV file into a DataFrame and report its shape, dtypes, and the output of `describe`.
2. Select all rows where one numeric column exceeds its mean, using boolean indexing.
3. Count the missing values in each column, then impute one numeric column with a justified strategy and state the reason.
4. Group the data by a categorical column and compute the sum and mean of a numeric column.
5. Merge two DataFrames with an inner join and again with a left join, and explain the difference in the result.
6. Reshape a DataFrame from wide to long format with `melt`, then back with `pivot`.
7. Produce a descriptive report on a real dataset and write a paragraph interpreting the findings.